Learning Document

Hyperparameters, Tuning and GridSearchCV

With Detailed Examples, Explained Code and Mini Project 3 Usage

Focus: Mini Project 3 - Multi-Algorithm Recommendation System Comparison

This handout explains what hyperparameters are, why tuning is needed, how GridSearchCV works, and how students should apply it in Mini Project 3.

Contents

1. What is a Hyperparameter?

A hyperparameter is a setting that we choose before training a machine learning model. The model does not learn this value automatically from the data. Instead, we provide the value to control how the algorithm learns.

Simple meaning

A hyperparameter is a model setting decided by the developer before training. It controls the behavior, flexibility or learning style of the model.

Parameter vs Hyperparameter

Item	Parameter	Hyperparameter
Who decides it?	Learned automatically by the model	Set by the developer before training
When is it created?	During model training	Before model training
Example in Linear Regression	Coefficient and intercept	No major tuning parameter in basic form
Example in Ridge Regression	Coefficient values	alpha
Example in Logistic Regression	Weights for each feature	C, solver, max_iter
Example in K-Means	Cluster centers after training	n_clusters

Easy Example

Imagine training a student for an exam.

Concept	Real-life Meaning	ML Meaning
Parameter	Marks learned after studying and writing tests	Values learned by the model from training data
Hyperparameter	Study hours, number of practice tests, revision method	Settings chosen before training the model

Examples of Hyperparameters

Algorithm	Hyperparameter	What It Controls
Ridge Regression	alpha	Strength of regularization. Higher alpha reduces overfitting but may underfit.
Logistic Regression	C	Inverse of regularization strength. Smaller C means stronger regularization.
Logistic Regression	solver	Optimization method used to find the best model weights.
Logistic Regression	max_iter	Maximum number of training iterations allowed.
K-Means	n_clusters	Number of customer groups or clusters to create.
Decision Tree	max_depth	Maximum depth of the tree. Controls model complexity.

2. What is Hyperparameter Tuning?

Hyperparameter tuning is the process of testing different hyperparameter values and selecting the combination that gives the best model performance. It helps us avoid guessing model settings manually.

> **Simple meaning**
>
> Tuning means trying multiple model settings, comparing the results, and selecting the best-performing setting.

Why Do We Tune Hyperparameters?

- Default model settings may not be the best for our dataset.
- A model may overfit if it is too complex.
- A model may underfit if it is too restricted.
- Tuning helps improve accuracy, prediction quality and business usefulness.
- It gives evidence for why one model setting was selected over another.

Overfitting and Underfitting

Condition	Meaning	Example
Underfitting	Model is too simple and performs poorly on both training and test data.	Ridge alpha is too high, so the model becomes too restricted.
Good fit	Model learns useful patterns and performs well on test data.	Ridge alpha is balanced.
Overfitting	Model memorizes training data and performs poorly on new data.	Regularization is too weak or model is too complex.

Manual Tuning vs GridSearchCV

Approach	How It Works	Problem / Benefit
Manual tuning	Developer tries one value at a time.	Slow and may miss the best combination.
GridSearchCV	Automatically tests all values from a predefined grid.	Systematic and beginner-friendly.
RandomizedSearchCV	Randomly tests selected combinations from a large search space.	Faster for large parameter spaces.

3. What is GridSearchCV?

GridSearchCV is a Scikit-learn technique used to find the best hyperparameter values for a model. It combines two ideas: grid search and cross-validation.

Grid Search

Grid search means trying every possible combination from a predefined list of values.

```
param_grid = {
    'alpha': [0.01, 0.1, 1, 10, 100]
}
```

For Ridge Regression, the above grid means GridSearchCV will train and compare Ridge models using alpha values 0.01, 0.1, 1, 10 and 100.

Cross-Validation

CV means cross-validation. Instead of validating the model only once, the training data is divided into multiple folds. The model is trained and validated multiple times, and the average score is used.

If cv = 5	Training Data	Validation Data
Round 1	Fold 2, 3, 4, 5	Fold 1
Round 2	Fold 1, 3, 4, 5	Fold 2
Round 3	Fold 1, 2, 4, 5	Fold 3
Round 4	Fold 1, 2, 3, 5	Fold 4
Round 5	Fold 1, 2, 3, 4	Fold 5

Why cross-validation is useful

It gives a more reliable score than one train-test split because the model is validated on different parts of the data.

Basic GridSearchCV Syntax

```
from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy'
)

grid_search.fit(X_train, y_train)
```

Important GridSearchCV Arguments

Argument	Meaning
estimator	The machine learning model, such as Ridge or LogisticRegression.
param_grid	Dictionary of hyperparameters and values to test.
cv	Number of cross-validation folds.
scoring	Metric used to select the best model.
n_jobs	Number of CPU cores to use. Use -1 to use all available cores.
verbose	Controls how much training progress is printed.

4. Complete Sample Dataset Code

The following sample code creates a small synthetic e-commerce dataset. Students can replace this with a real dataset later. This is useful for practice because the code can run without downloading any dataset.

```
import numpy as np
import pandas as pd

np.random.seed(42)

n = 500

data = pd.DataFrame({
    'price': np.random.randint(100, 5000, n),
    'views': np.random.randint(1, 80, n),
    'time_spent': np.random.randint(10, 600, n),
    'previous_purchases': np.random.randint(0, 20, n),
    'cart_status': np.random.randint(0, 2, n),
    'product_category': np.random.choice(['Fashion', 'Electronics', 'Home', 'Beauty'], n)
})
```

```
# Rating is created using a simple pattern plus random noise.
data['rating'] = (
    2.5
    + 0.03 * data['previous_purchases']
    + 0.002 * data['views']
    + 0.0005 * data['time_spent']
    - 0.00005 * data['price']
    + np.random.normal(0, 0.4, n)
)

data['rating'] = data['rating'].clip(1, 5)

# Purchase status is created as a binary target.
data['purchase_status'] = (
    (data['cart_status'] == 1) &
    (data['time_spent'] > 150) &
    (data['previous_purchases'] > 2)
).astype(int)

print(data.head())
```

Explanation of the Dataset Code

Code Part	Explanation
np.random.seed(42)	Makes the random data reproducible. Every run gives the same sample data.
n = 500	Creates 500 customer-product interaction records.
price, views, time_spent	Numerical features that describe user behavior and product information.
product_category	Categorical feature that must be encoded before model training.
rating	Regression target. The model will learn to predict this value.
purchase_status	Classification target. 1 means purchased, 0 means not purchased.

Shared Preprocessing Setup

For real projects, preprocessing should be placed inside a Pipeline. This prevents data leakage because scaling and encoding are learned only from the training folds during cross-validation.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

features = [
    'price', 'views', 'time_spent',
    'previous_purchases', 'cart_status', 'product_category'
]

X = data[features]
```

```
numeric_features = ['price', 'views', 'time_spent', 'previous_purchases', 'cart_status']
categorical_features = ['product_category']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ]
)
```

Explanation of Preprocessing Code

Code Part	Explanation
StandardScaler	Scales numerical columns so that large values like price do not dominate smaller values.
OneHotEncoder	Converts text categories like Fashion and Electronics into numeric columns.
handle_unknown="ignore"	Prevents errors if a new category appears in test data.
ColumnTransformer	Applies different preprocessing steps to numerical and categorical columns.

5. Example 1: GridSearchCV with Ridge Regression

Problem

Predict the rating a user may give to a product. This is a regression problem because the target value is numeric.

What to Tune

Hyperparameter	Meaning	Values to Try
alpha	Controls regularization strength.	0.01, 0.1, 1, 10, 100

Ridge Regression Code

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Target for regression
y_rating = data['rating']

X_train, X_test, y_train, y_test = train_test_split(
    X, y_rating, test_size=0.2, random_state=42
)

ridge_pipeline = Pipeline(steps=[
```

```
    ('preprocessor', preprocessor),
    ('model', Ridge())
])

ridge_param_grid = {
    'model__alpha': [0.01, 0.1, 1, 10, 100]
}

ridge_grid = GridSearchCV(
    estimator=ridge_pipeline,
    param_grid=ridge_param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1
)

ridge_grid.fit(X_train, y_train)

print('Best Ridge Parameters:', ridge_grid.best_params_)
print('Best CV R2 Score:', ridge_grid.best_score_)

best_ridge_model = ridge_grid.best_estimator_
y_pred = best_ridge_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print('MAE:', mae)
print('RMSE:', rmse)
print('R2 Score:', r2)
```

Explanation of Ridge Code

Line / Part	Explanation
y_rating = data["rating"]	Selects the target column for regression.
train_test_split	Splits data into training and testing sets. The test set is kept unseen until final evaluation.
Pipeline	Combines preprocessing and Ridge Regression into one workflow.
model__alpha	Because Ridge is inside the pipeline under the name model, the hyperparameter is written as model__alpha.
cv=5	Uses 5-fold cross-validation during tuning.
scoring="r2"	Chooses the alpha value with the best R2 score.
best_estimator_	Stores the complete best pipeline including preprocessing and model.

How to Interpret Ridge Results

Metric	Meaning	Good Result
MAE	Average absolute prediction error.	Lower is better.
RMSE	Error metric that penalizes larger mistakes more strongly.	Lower is better.
R2 Score	Shows how much target variation is explained by the model.	Closer to 1 is better.

Business interpretation

If Ridge Regression predicts ratings accurately, the e-commerce system can recommend products that customers are more likely to rate highly.

6. Example 2: GridSearchCV with Logistic Regression

Problem

Predict whether a user is likely to purchase a product. This is a classification problem because the target value is binary: purchased or not purchased.

What to Tune

Hyperparameter	Meaning	Values to Try
C	Inverse of regularization strength. Smaller C means stronger regularization.	0.01, 0.1, 1, 10
solver	Optimization algorithm used by Logistic Regression.	liblinear, lbfgs
max_iter	Maximum number of iterations for convergence.	200, 500, 1000

Logistic Regression Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

# Target for classification
y_purchase = data['purchase_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y_purchase, test_size=0.2, random_state=42, stratify=y_purchase
)

logistic_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', LogisticRegression())
])
```

```
logistic_param_grid = {
    'model__C': [0.01, 0.1, 1, 10],
    'model__solver': ['liblinear', 'lbfgs'],
    'model__max_iter': [200, 500, 1000]
}

logistic_grid = GridSearchCV(
    estimator=logistic_pipeline,
    param_grid=logistic_param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)

logistic_grid.fit(X_train, y_train)

print('Best Logistic Parameters:', logistic_grid.best_params_)
print('Best CV F1 Score:', logistic_grid.best_score_)

best_logistic_model = logistic_grid.best_estimator_
y_pred = best_logistic_model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred, zero_division=0))
print('Recall:', recall_score(y_test, y_pred, zero_division=0))
print('F1 Score:', f1_score(y_test, y_pred, zero_division=0))
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
```

Explanation of Logistic Regression Code

Line / Part	Explanation
y_purchase = data["purchase_status"]	Selects the binary target for classification.
stratify=y_purchase	Keeps the same purchase/non-purchase ratio in train and test data.
model__C	Tunes regularization strength inside the pipeline.
scoring="f1"	Useful when both precision and recall matter.
zero_division=0	Avoids warnings if the model predicts no positive class in a small dataset.
classification_report	Shows precision, recall and F1 score for each class.

How to Interpret Classification Results

Metric	Meaning	Business Meaning
Accuracy	Overall correct predictions.	Useful if purchased and not-purchased classes are balanced.
Precision	Out of predicted buyers, how many really purchased.	Important when marketing offers should not be wasted.

Recall	Out of actual buyers, how many were found by the model.	Important when the business wants to capture most potential buyers.
F1 Score	Balance between precision and recall.	Good single metric when data is imbalanced.
Confusion Matrix	Shows true positives, false positives, true negatives and false negatives.	Helps understand where the model makes mistakes.

Business interpretation

If Logistic Regression predicts purchase likelihood well, the business can target likely buyers with coupons, reminders, offers or product recommendations.

7. Example 3: Tuning K-Means Clustering

K-Means is an unsupervised learning algorithm. It does not use a target column. For Mini Project 3, it can be used to segment customers based on behavior.

Important Hyperparameter

Hyperparameter	Meaning
n_clusters	The number of customer groups the algorithm should create.

Important note

GridSearchCV is mainly used for supervised learning models. For K-Means, it is better for beginners to use the Elbow Method and Silhouette Score to choose n_clusters.

K-Means Code

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
import pandas as pd

cluster_features = [
    'price', 'views', 'time_spent',
    'previous_purchases', 'cart_status'
]

X_cluster = data[cluster_features]

scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)

results = []

for k in range(2, 8):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_cluster_scaled)
```

```
    inertia = kmeans.inertia_
    silhouette = silhouette_score(X_cluster_scaled, labels)

    results.append({
        'k': k,
        'inertia': inertia,
        'silhouette_score': silhouette
    })

cluster_results = pd.DataFrame(results)
print(cluster_results)

best_k = cluster_results.sort_values('silhouette_score', ascending=False).iloc[0]['k']
print('Best k based on Silhouette Score:', best_k)
```

Explanation of K-Means Code

Line / Part	Explanation
cluster_features	Selects behavior-related columns for customer segmentation.
StandardScaler	Scaling is important because K-Means uses distance.
for k in range(2, 8)	Tests different cluster counts from 2 to 7.
inertia_	Measures how compact the clusters are. Lower is better, but it always decreases as k increases.
silhouette_score	Measures how well-separated the clusters are. Higher is better.
best_k	Chooses the k value with the highest silhouette score.

How to Explain Customer Segments

Cluster	Possible Customer Meaning	Business Action
Cluster 0	High engagement and frequent purchase behavior.	Recommend premium products and loyalty offers.
Cluster 1	High views but low purchases.	Use cart reminders, discounts or trust-building messages.
Cluster 2	Low engagement customers.	Use awareness campaigns and simple product recommendations.
Cluster 3	Price-sensitive customers.	Show budget-friendly products and offers.

8. How to Compare Results in Mini Project 3

Students should not only run models. They should compare the output and explain which model is useful for which business purpose.

ML Task	Algorithm	Target / Goal	Tuning Method	Metrics
Regression	Ridge Regression	Predict user rating	GridSearchCV on	MAE, RMSE, R2

			alpha	
Classification	Logistic Regression	Predict purchase likelihood	GridSearchCV on C, solver, max_iter	Accuracy, Precision, Recall, F1
Clustering	K-Means	Segment customers	Elbow Method and Silhouette Score on n_clusters	Inertia, Silhouette Score

Sample Final Comparison Table

Task	Best Setting	Best Metric Result	Business Interpretation
Rating Prediction	alpha = 10	R2 = 0.82	Model can recommend products likely to receive higher ratings.
Purchase Prediction	C = 1, solver = liblinear	F1 = 0.76	Model can identify likely buyers for targeted offers.
Customer Segmentation	k = 4	Silhouette = 0.51	Customers can be grouped into meaningful marketing segments.

How Students Should Write the Analysis

- Mention the algorithm used and why it fits the problem.
- Mention the hyperparameters tuned and the values tested.
- Mention the best hyperparameter values selected by GridSearchCV or silhouette score.
- Compare before-tuning and after-tuning performance if possible.
- Explain the result in business language, not only technical language.

9. Common Mistakes to Avoid

Mistake	Why It Is a Problem	Correct Approach
Tuning on test data	The test data becomes exposed to the model selection process.	Use GridSearchCV only on training data.
No preprocessing pipeline	Scaling or encoding may leak information from validation folds.	Use Pipeline and ColumnTransformer.
Using accuracy only	Accuracy can be misleading when purchase data is imbalanced.	Use F1, precision and recall.
Too many hyperparameter values	GridSearchCV becomes slow.	Start with a small, meaningful grid.
Ignoring business meaning	The report becomes only technical.	Explain how the model helps recommendations, offers or segmentation.

10. Mini Project 3 Final Checklist

Checklist Item	Expected Evidence in Submission
Dataset overview	Columns, rows, target columns and feature description.
Regression model	Ridge Regression code, alpha tuning and regression metrics.
Classification model	Logistic Regression code, hyperparameter tuning and classification metrics.
Clustering model	K-Means code, tested k values, silhouette score and customer segment interpretation.
GridSearchCV explanation	Short explanation of grid search, cross-validation and best_params_.
Comparison table	Final table comparing algorithms, settings, metrics and business use.
Conclusion	Clear recommendation on which model supports which business goal.

11. Quick Revision Summary

Term	Simple Meaning	Mini Project 3 Example
Hyperparameter	A model setting chosen before training.	alpha in Ridge Regression, C in Logistic Regression.
Tuning	Trying different settings to find the best model.	Testing alpha values 0.01, 0.1, 1, 10 and 100.
GridSearchCV	A method that tests all given settings using cross-validation.	Selects the best alpha or C value.
Cross-validation	Training and validating multiple times on different folds.	cv=5 means 5 validation rounds.
Pipeline	Combines preprocessing and model training safely.	Scaling, encoding and Ridge Regression together.

Final takeaway

Hyperparameter tuning helps students choose model settings based on evidence instead of guessing. GridSearchCV is a structured and beginner-friendly way to tune supervised ML models in Mini Project 3.